



Figure 3: The estimated “horizon-average” treatment effects (ATT_h , Equation 3) 0 to +6 months after adoption, plus the “placebo” pre-adoption treatment effect estimates ($\hat{\tau}_h$ for testing the parallel trend assumption, Equation 5) -6 to -2 months before adoption. All outcome variables are log-transformed same as the estimated average treatment effects in Table 2.

outcome variables pass the heteroscedasticity- and cluster-robust Wald tests [29] at the 0.05 level (i.e., passing pre-trend tests).

4.1.1 Development Velocity. On average, Cursor adoption has a modestly significant positive impact on development velocity, particularly in terms of code production volume: Lines added increase by about 28.6% (Table 2). There is no statistically significant effect for the volume of commits. The horizon-average treatment effect estimations (Figure 3) reveal important temporal patterns that explain these differences: *The only significant development velocity gain is in the first two months post Cursor adoption.* The models estimate a 55.4% increase in commits in the first month, a 14.5% increase in commits in the second month, a 281.3% increase in lines added in the first month, and a 48.4% increase in lines added in the second month, respectively.

4.1.2 Software Quality. In contrast to the transient velocity gains, Cursor adopters show sustained patterns across static analysis warnings and code complexity. On average (Table 2), static analysis warnings increase significantly by 30.3%, and code complexity increases by 41.6%. The effect on duplicate line density is insignificant. The horizon-average treatment effect estimates (Figure 3) reveal that the increase in the two outcome metrics persists beyond the initial adoption period, contrasting the transient velocity gains.

4.2 Velocity & Quality Interactions

To distangle the temporal interactions between velocity and quality, we summarize dynamic panel GMM models testing causal paths specified in Equation 7, in Table 3. All models pass the Sargan test (confirming instrument validity) and AR(2) test (confirming no serial correlation in the original errors), validating the moment conditions required for causal interpretation [22].

The first two models show that, on average (across all Cursor-adopters and non-adopters in our dataset), and *holding all other temporal dynamic factors constant*: (1) An increase in development velocity does not produce a significant effect on static analysis warnings and code complexity. (2) Cursor adoption does not have a significant effect on static analysis warnings. Notably, increases in codebase size are a major determinant of increases in static analysis warnings and code complexity, and absorb most variance in the two outcome variables. However, even with strong controls for codebase size dynamics, the adoption of Cursor still has a significant effect

Table 3: The dynamic panel GMM estimates testing temporal interactions between velocity and quality attributes. L , W , and C stand for lines added, static analysis warnings, and code complexity, respectively (see Equation 7). The estimates for the remaining covariates are omitted here for brevity.

	$L_{it} \rightarrow W_{it}$	$L_{it} \rightarrow C_{it}$	$C_{it} \rightarrow L_{i,t+1}$	$W_{it} \rightarrow L_{i,t+1}$
Main Effect	-0.000 (0.015)	-0.006 (0.016)	-0.718*** (0.098)	-0.588*** (0.092)
Cursor	-0.011 (0.033)	0.086** (0.030)	1.044*** (0.124)	1.048*** (0.124)
Lines of Code	0.845*** (0.073)	0.852*** (0.059)	0.869*** (0.153)	0.851*** (0.155)
Num. Obs.	14,755	14,755	14,755	14,755
Sargan p	0.248	0.141	0.633	0.639
AR(1) p	<0.001	<0.001	<0.001	<0.001
AR(2) p	0.734	0.438	0.393	0.330

Notes: Two-way fixed effects (repository + month), two-step GMM with first-difference transformation. Robust standard errors in parentheses. *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$. The contemporaneous variables L_{it} , C_{it} , and W_{it} are instrumented with lags 2-3 to address endogeneity. Sargan $p > 0.05$ indicates valid instruments. AR(1) $p < 0.05$ is expected with the first-difference transformation. AR(2) $p > 0.05$ indicates no serial correlation in the original errors. Sargan $p > 0.05$ and AR(2) $p > 0.05$ validate the moment conditions required for causal interpretation [22].

on code complexity, leading to a 9% baseline increase on average compared to projects in similar dynamics but not using Cursor.

The last two models show that, on average, and *holding all other temporal dynamic factors constant*: (1) A 100% increase in code complexity and static analysis warnings causes a 64.5% and 50.3% decrease in development velocity as measured by lines added, respectively. (2) The adoption of Cursor results in a 1.84x baseline increase in lines added post adoption. Thus, the velocity gain from Cursor adoption would be fully cancelled out by a ~5x increase in static analysis warnings or a ~3x increase in code complexity, according to the dynamic panel GMM estimations.

To summarize, the dynamic panel GMM models suggest that: (1) *The adoption of Cursor leads to an inherently more complex codebase;* and (2) *The accumulation of static analysis warnings and code complexity decreases development velocity in the future.*